Forecasting Privacy-Budget Consumption in Repeated DP-SQL Workloads

Can Öztaş
Hacettepe University
Ankara, Turkey
canoztas06@gmail.com

Engin Demir
Hacettepe University
Ankara, Turkey
engindemir@hacettepe.edu.tr

Abstract

Differentially private (DP) SQL systems add calibrated noise to each aggregate query and charge it against a finite privacy budget; once the budget is exhausted, no further query can be answered. Existing systems account for each query in isolation, although workloads repeat heavily and an exact repeat is free by post-processing. A data custodian therefore cannot tell in advance whether a workload will fit its budget. We present a closed-form model that forecasts a workload's budget consumption before any query executes, from public repetition structure alone: the distribution over distinct queries, the workload length, and the budget. The model yields the expected spend, a hard completion guarantee under a safe per-query budget, and a per-release accuracy verdict. Being mechanism-agnostic, it extends to Markovian and regime-switching arrivals and transfers unchanged to multi-table joins. The forecast is the accounting core of a DP-SQL middleware we implement over DuckDB and PostgreSQL: it sizes the budget allocator, gates temporal re-releases, and prices equivalence-aware caching. Across a 4,155-trial campaign the forecast matches observed consumption within 3% in 21 of 24 configurations, and on 30 workloads derived from real Amazon Redshift traces, forecasting from the first half of each workload reduces the budget-prediction error by 45%.

1 Introduction and Motivation

Aggregate statistics computed over sensitive data leak information about the individuals they summarize. A pair of COUNT queries that differ by one predicate reveals an individual's attribute by differencing; repeated over a dataset, such queries reconstruct it [3]. Differential privacy (DP) [1, 2] is the standard defense: perturb each answer with noise calibrated to the query's sensitivity and a per-query budget ε_q, and bound the cumulative privacy loss by composition. The budget is a hard, non-renewable resource: once the total $\sum \varepsilon_q$ reaches a cap B, no further query can be answered without breaking the guarantee. A data custodian therefore needs to understand, and ideally predict, *how fast a workload spends its budget.*

The need for such a prediction is best illustrated with a concrete scenario. Consider an analytics team that publishes a privacy-protected dashboard: a fixed set of tiles (counts, sums, group-by breakdowns) refreshed on a schedule, plus occasional ad-hoc drill-downs. The custodian sets a monthly budget B and a per-query ε_q. Two failures follow from treating queries in isolation. If ε_q is set to meet an accuracy target, naive composition exhausts B after only B/ε_q releases, and the dashboard can no longer be refreshed for the remainder of the month. If ε_q is set small enough to survive the month, every tile is needlessly noisy. The custodian cannot tell in advance which will happen, because no model relates the *shape* of the workload (how much it repeats) to the budget it will spend. That repetition is precisely the structure our model exploits: the same dashboard tiles recur, so the budget is governed by the handful of *distinct* templates rather than by the hundreds of refreshes.

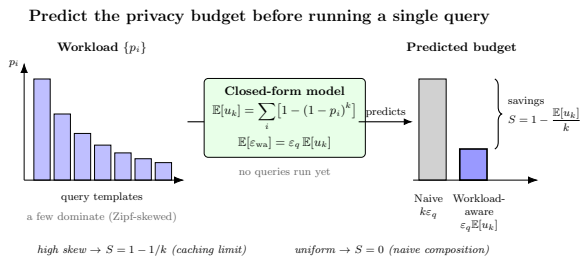

Figure 1: Naive composition charges every one of k queries; workload-aware accounting charges only the u_k distinct queries (a template together *with* its WHERE literals), since exact repeats are served from cache by post-processing. The model predicts $\mathbb{E}[u_k]$, and hence the budget, before execution.

Existing systems do not exploit this repetition. Most DP-SQL systems (PINQ [4], Chorus [6], DOP-SQL [7]) charge every query a fresh ε_q and account for it in isolation (PrivateSQL [5] instead amortizes a one-time synopsis budget—Section 2—but still fixes that budget up front rather than forecasting it). Yet analytical workloads are highly repetitive. A monitoring dashboard re-issues the *exact same* key performance indicator (KPI) tile on a schedule (a canonicalized exact-query repeat, served free), whereas a drill-down reuses a structural template with *different* literals (a distinct exact query that is charged afresh). Only exact repeats save budget; the savings come from how often a dashboard re-issues identical queries. Under exact-repeat caching, a repeated answer is free: re-using an already-released noisy value adds no privacy cost, since any function of a DP output is itself DP (DP's *post-processing* property). Consequently, the budget a workload of length k consumes is governed not by k but by its number of *distinct* queries u_k (Figure 1). The central question of this project is whether u_k, and hence the budget, can be *predicted from the workload's repetition structure* in closed form, before the workload runs. Exact-repeat caching itself is a known mechanism and we claim no novelty for it; our contribution is *forecasting* its budget impact *before* execution.

Contributions. This work makes three contributions.

(1) **A before-execution budget forecast for repeated DP-SQL workloads**, to our knowledge the first that is *closed-form* and reads only public structure—the distinct-query

distribution $\{p_i\}$, the workload length k, and the budget B—never touching the data. Before any query executes, the forecast tells a data custodian how much of the budget the workload will consume and whether the workload fits within it. It delivers three outputs of differing strength. The first is the *expected* spend $\varepsilon_q \mathbb{E}[u_k]$, an average-case estimate. The second, under the safe sizing $\varepsilon_q = B/m$, is a *hard* completion guarantee, since $u_k \le m$ in the static regime. The third is a *per-release* (α, β)-accuracy verdict: each release *individually* has error at most α with probability $\ge 1 - \beta$; this is not a joint guarantee that every release succeeds simultaneously (Section 3). No existing DP-SQL system natively exposes such a workload-level forecast (Section 2). Five propositions establish this core model (expected distinct count, savings, the Zipf-skew case, a concentration bound, and per-query utility), with two limits recovering the folklore $1 - 1/k$ caching rule and standard per-query accounting; six further propositions extend it to temporal re-release, Markovian and regime-switching arrivals, and joins (eleven in total).

(2) **A measured forecasting improvement on production-derived traces.** On 30 Redbench workloads (synthesized from real Amazon Redshift query traces), forecasting the full distinct-query count from only the *first half* of each workload—via an unseen-species correction—cuts the normalized budget-prediction error, measured as mean absolute error (MAE), by 45% ($0.22 \to 0.12$) relative to the plug-in estimate (Section 4.2).

(3) **A workload-aware middleware architecture and a validation campaign.** The forecast is not an add-on to an existing cache but the accounting core of the middleware: the same closed form sizes the budget allocator (the safe $\varepsilon_q = B/m$ and the predictive $\varepsilon_q = B/\hat{U}$), gates temporal re-releases (Proposition 7), and prices equivalence classes rather than query strings. We instantiate this architecture over DuckDB and PostgreSQL (equivalence-aware caching, a temporal staleness/update extension, and top-k as DP post-processing) and validate the forecast on a 4,155-trial campaign—within $< 3\%$ in 21/24 cells, invariant to data scale and to ε_q, and exact on a constructed mixed workload—with two leakage experiments confirming the per-query calibration. For reproducibility, the complete implementation, the 233-test suite, and a generator script for every reported table and figure are publicly available with fixed random seeds, so that all results can be regenerated from a fresh checkout (Section 7).

We do *not* claim to outperform deployed DP systems. The 100× saving over naive composition is a secondary demonstration of the mechanism, the 1.8× advantage over a learned bandit is an allocator ablation, and the savings we achieve against reactive DP caches (Turbo, CacheDP) are *on par* with theirs. The contribution is the before-execution forecast they do not natively expose (Section 4). The headline grid is a self-consistency check (Section 7); the predictive allocator and the learned-allocator comparison are presented in Appendix A, and the semantic-caching negative result in Appendix D.

2 Related Work

Prior work relevant to our forecast falls into four lines: DP-SQL systems, workload-aware DP and caching, multi-query accounting and online budget allocation, and distinct-count estimation. We review each in turn.

The first line consists of DP-SQL systems. PINQ [4] introduced composable DP query operators; Chorus [6] rewrites SQL to enforce DP; DOP-SQL [7] tunes per-query mechanisms. Each exposes a *fixed* per-query ε_q chosen up front and accounts for queries in isolation. PrivateSQL [5] is different in kind: it spends a one-time budget building DP synopses (private materialized views) over a *representative* workload, then answers subsequent queries from those synopses with *no* additional privacy loss. This is amortization across a declared query set rather than per-query accounting. None of them *forecasts* a repeated workload's aggregate consumption from its repetition structure. PrivateSQL, for instance, fixes its synopsis budget up front from a representative set but emits no closed-form, before-execution prediction of the savings ratio $S(k)$ (formalized in Section 3) for an arbitrary repeated stream.

A second line makes the DP mechanism workload-aware or adds a cache. The matrix mechanism [10] optimizes noise for a *known* batch of linear queries, and Dong, Sun, and Yi [11] beat worst-case composition by exploiting relational structure. DP caches make repeated work cheaper at runtime: Turbo [13] conserves 1.7–15.9× budget by generalizing across overlapping linear queries, and CacheDP [14] answers a workload at lower budget under an accuracy contract. Two *proactive* systems are closest to our setting. Pioneer [30] reused past noisy results to answer a new query more cheaply, sometimes for free, but by its authors' own statement it optimizes only the *current* query and leaves future-workload effects open; LAPRAS [29] precomputes an offline batch mechanism on a *predicted* query set before the stream arrives and serves it at no extra cost—but it needs a concrete predicted set and *touches the sensitive data* ahead of time, whereas we forecast in closed form from *public* repetition structure $(\{p_i\}, k, B)$ alone, without touching the data. All of these systems deliver the *savings* our model prices, but they do so reactively (Turbo, CacheDP, Pioneer) or by precomputing on the data (LAPRAS), and none provides a closed-form structural forecast. Our exact-repeat caching is a simple special case, and the mechanism itself carries no novelty claim (Section 4).

A third line addresses multi-query accounting and online budget allocation. Tight composition—the moments accountant [15], Rényi DP [16], and zero-concentrated DP (zCDP) [17]—reduces the *rate* of budget spend and is orthogonal to our *count* of distinct releases. APEx [8] translates a per-query accuracy target into the minimum ε at runtime, one query at a time; BGTplanner [9] learns a Gaussian-process-plus-bandit budget policy for federated-learning rounds, driven by an observed-utility signal. Both decide budget reactively; our forecast instead comes from workload structure. A related family of methods *schedules* a known budget across competing tasks or analysts; it includes privacy-budget scheduling with its fairness variants, as well as multi-analyst budget sharing [12], which invokes the coupon-collector problem to bound the number of queries each analyst may issue (a throughput limit across *analysts*, not a forecast of the number of distinct queries). This scheduling line has recently been consolidated into an explicit systems doctrine that

Table 1: Positioning. To our knowledge, no prior system natively exposes a closed-form, before-execution budget forecast from public workload structure alone.

System	workload model	predict budget	exact-repeat free
PINQ / Chorus / DOP-SQL	–	no	no
PrivateSQL	repr. batch	no	synopsis
Turbo / CacheDP	–	no	yes (reactive)
Pioneer / LAPRAS	predicted set	no (precompute, on data)	partial
APEx	–	no (per-query)	no
BGTplanner	learned GP	no	no
Ours	**closed-form**	**yes (a-priori)**	**yes**

treats privacy as a first-class computing resource, managed with schedulers, quotas, admission control, and caches, from the browser to the datacenter [37]. We adopt the same resource view and supply the input that resource management presumes but these systems take as given: all of them consume per-task demands, whereas our model *predicts* the demand, in closed form, before execution.

What none of these lines provides is a before-execution workload forecast. The requirement can be stated as a single task: at t=0, before any query runs and before any budget is spent, report, from only the public structure $(m, k, \{p_i\}, B)$, where m denotes the number of possible distinct queries, three *distinct* outputs: the *expected* spend $\varepsilon_q \mathbb{E}[u_k]$, a *hard* completion guarantee under the B/m sizing ($u_k \le m$), and a *per-release* (α, β)-accuracy verdict (each release individually, not jointly). None of the systems reviewed above natively performs this task. Turbo and CacheDP need cache state and already-executed answers; APEx converts a *single* query's accuracy target into an ε, with no model of workload arrivals or repetition; PrivateSQL builds a synopsis from a representative workload but does not forecast an arbitrary stream's spend; and even the proactive LAPRAS precomputes on a *predicted query set* and on the data, whereas we read only public structure. Our model produces all three directly from $(\{p_i\}, k, B)$ without touching the data. That is the narrow sense in which our model is, to our knowledge, the first. Our work sits at the intersection of two mature lines, DP-SQL accounting and unseen-species estimation, and borrows the occupancy mathematics and exact-repeat caching, claiming novelty for neither. The contribution is the link between them: a closed-form, before-execution forecast of $S(k)$ from $\{p_i\}$, with the limit, concentration, and utility consequences that follow.

The second of those mature lines, distinct-count estimation, supplies the mathematical core of our forecast. Predicting the number of distinct queries in a length-k workload is the classical unseen-species problem. The occupancy identity $\mathbb{E}[u_k] = \sum_i [1 - (1 - p_i)^k]$ is textbook [18]; horizon-aware extrapolators (Good–Toulmin [19] and its smoothed variant [20]) predict new species from a prefix, while richness estimators such as Chao1 [21] estimate total richness. This line has also been carried out under DP itself: INSPECTRE [31] estimates the unseen privately, but it targets *support coverage* rather than budget. We connect these tools to DP-*budget* forecasting; query-template forecasting itself (QueryBot 5000 [22]) is non-private.

Table 1 summarizes this positioning. In short, to our knowledge no prior DP-SQL system natively exposes, in closed form and *before execution*, a workload-level forecast from public repetition structure alone, separating the *expected* spend, the *hard* completion guarantee of the B/m sizing, and a *per-release* accuracy verdict. Providing that forecast, and measuring on real traces how accurately it can be made from a workload prefix, is the contribution of this project.

3 Methodology

The methodology has three parts, built on a single underlying idea. We place a thin *middleware* between the analyst and the database (Section 3.1): every incoming query is checked against the answers already released; an *exact repeat* is returned from cache for free, while a *new* query is given just enough random noise to hide any single row and is charged a slice of the privacy budget. Because identical queries cost nothing, the budget a workload spends is driven by how many *distinct* queries it contains—and hence by how much the workload repeats. The core of the section (Section 3.2) turns this observation into a handful of formulas that predict, *before any query runs*, how many distinct queries a workload will contain, how much budget it will therefore spend, how much that saves over noising every query separately, and how accurate the answers can be at a fixed budget. Each formula comes with the conditions under which it holds. We then extend the model to data that *changes over time*, where cached answers go stale and must occasionally be refreshed (Section 3.3). Privacy is never traded away for the forecast: the noise on each released answer stands on its own, and the model predicts only the *bookkeeping*, that is, how fast the budget is consumed.

3.1 Threat model and preliminaries

A relational dataset D (DuckDB or PostgreSQL) is queried through the middleware. The adversary is any analyst who can adaptively issue supported queries and observe the noisy outputs; the goal is to bound inference about any single tuple.

Definition 1 (Tuple-level neighbors). *$D \sim D'$ iff they differ in exactly one tuple (unbounded model).*

Definition 2 (ε-DP [1]). *$\mathcal{M}$ is ε-DP iff, for every neighboring pair D, D' and measurable S,*

$$\mathbb{P}[\mathcal{M}(D) \in S] \le e^{\varepsilon}\, \mathbb{P}[\mathcal{M}(D') \in S].$$

Removing or adding any one person's row may change the *probability* of any output by at most a factor e^{ε}, so no single individual visibly affects the answer; the parameter ε thus controls the privacy level, with smaller ε requiring more noise and giving stronger privacy. The *sensitivity* Δf of a query is how much one row can move its true answer (a COUNT moves by 1, a SUM by the column bound B_c); calibrating the noise to $\Delta f / \varepsilon_q$ is what buys the guarantee. Every query spends ε_q from a fixed budget B; when $\sum \varepsilon_q$ reaches B, no further query can be answered. That ceiling is exactly the resource this paper learns to forecast.

The middleware supports a restricted SQL fragment with explicitly bounded sensitivities: SELECT with COUNT, SUM, AVG, single-table FROM, conjunctive WHERE, and optional GROUP BY. Tuple-level sensitivities are $\Delta f = 1$ for COUNT, $\Delta f = B_c$ for SUM(c), and—conservatively—$\Delta f = B_c$ for AVG (a worst-case group of size one, avoiding the implicit n-leak of the empirical-mean mechanism, since n is itself private). The per-column bound B_c is a *public* schema-domain constant (e.g. from the TPC-H spec or the Adult schema); to make the sensitivity hold *by construction* rather than by

assumption, the middleware *clamps* each summed/averaged value to $[0, B_c]$ inside the executed SQL (via GREATEST/LEAST, with SQL NULLs preserved), so every row's realized contribution is at most B_c even on out-of-domain data. The parser enforces a strict positive whitelist: any construct that could amplify a row's contribution or bypass the sensitivity bound—an aggregate wrapped in arithmetic or a window (e.g. $1000 \cdot$ SUM, COUNT OVER), common table expressions (CTEs) and set operations (a self-union doubles each row), derived tables, OFFSET, and `LIMIT` without an aggregate `ORDER BY`—is rejected *before* any budget is spent.

Given these sensitivities, each answer is released through the Laplace mechanism, and releases compose as follows. For sensitivity Δf and budget ε_q the middleware returns $\tilde{f}(D) = f(D) + \eta$, $\eta \sim \mathrm{Lap}(\Delta f/\varepsilon_q)$, where $\mathrm{Lap}(b)$ denotes the zero-mean Laplace distribution with scale b. A GROUP BY with g groups noises each group independently; since a tuple affects at most one group, *parallel composition* gives total cost ε_q, not $g\varepsilon_q$. This charges the aggregate *values* only and is ε_q-DP *provided the group-key domain is public and fully enumerated*, so that groups absent from the data are still released with noised zero counts. This requirement is substantive rather than a formality: returning only the keys *present* in the data, which the current prototype does, leaks membership through a singleton group (a secret degree value present in one record would appear in the output even at small ε). Therefore, all GROUP BY / top-k guarantees in the current prototype assume a *public and fully enumerated* key domain (e.g. the fixed `education` levels, with noised zeros for absent groups). This is a scope boundary of the prototype rather than a defect of the model. Lifting it for sensitive or unbounded key domains needs a stability-based threshold that suppresses low-count noisy groups, which we leave to future work. Multi-aggregate queries split ε_q evenly across aggregates by sequential composition. The exact-repeat cache returns a previously released answer with zero budget cost, since it is post-processing.

Beyond single aggregates, the middleware supports top-k queries (`... GROUP BY` g `ORDER BY agg LIMIT` k), handled entirely as post-processing, with one essential design point: ordering and truncation act on the *noised* histogram, never the true one. The engine fetches all g groups, noises every count under the same parallel composition (cost ε_q, *not* $k\varepsilon_q$), and only then sorts by the noisy value and keeps the top k. The raw `ORDER BY/LIMIT` is stripped before it reaches the database, since letting the DB apply it would select *or order* the reported groups on their *true* counts and leak which groups crossed the cutoff. The same applies to a bare `ORDER BY` *without* a `LIMIT`: it too is ranked on the noised values (true-count row order would otherwise leak the full ranking), and a `LIMIT` *without* an `ORDER BY` is rejected at parse time as an ill-defined selection. Because the released slate and its counts are a deterministic function of the ε_q-DP noisy histogram, top-k is post-processing and charges no extra budget (a report-noisy-max guarantee covering the whole top-k list at the cost of one histogram). The price is that selection is *approximate*: groups whose true counts straddle the k-th boundary may swap. The forecast and cache apply unchanged—a repeated top-k query is one template, free on exact repeat and counted once in $\mathbb{E}[u_k]$.

Finally, the forecast rests on two assumptions: (i) a *non-adaptive* analyst whose template choices are independent of the released noise, and (ii) that the template count m and distribution $\{p_i\}$ are public properties of the *query stream*, not of the protected rows. Privacy holds regardless (composition is adaptive-safe and total spend is hard-capped at B); the assumptions concern only the average-case *forecast* (Section 7).

3.2 The analytical model

Throughout, the privacy budget is the *resource* and the distinct-query count is the *structural proxy* that prices it: the forecast predicts the proxy, and the accountant converts it to spend at ε_q per distinct release (in privacy-odometer terms, we forecast the number of increments, not their size). The unit of budget accounting is the *exact* query—a structural template together with its WHERE literals, i.e. the cache key—since only a *canonicalized exact-query* repeat is served free (the key is the canonical SQL plus typed literals, so it is robust to whitespace and literal-format differences). Let m be the number of distinct exact queries the analyst may issue and $\{p_i\}_{i=1}^{m}$ a distribution over them; the analyst draws k queries independently and identically distributed (i.i.d.) from $\{p_i\}$, and u_k is the number of these distinct exact queries that appear at least once. (Two queries sharing a structural template but differing in literals are distinct queries, and each is charged.)

The unit of accounting can, however, be broadened beyond the byte-identical string. To this end, we pair the cache with a sound equivalence check (Section 5, Appendix D): a predicate-normalization layer plus an optional satisfiability-modulo-theories (SMT) layer backed by the Z3 solver reuses the noised answer for any *provably equivalent* query, so `age>=30` and `age>29`, or `age*2>=60` and `age>=30`, are counted as one distinct query rather than two. The check admits a match only when equivalence is proven (zero false collisions over 60,000+ adversarial predicate pairs), so reuse is always correct. With this layer the unit of accounting is the *equivalence class* of the query, and u_k counts *semantically* distinct queries, so the forecast prices equivalence classes, not exact strings. The model is unchanged: $\{p_i\}$ is read over classes and every result below applies verbatim. This makes the forecast *equivalence-aware*, the cache-key analogue of the forecast's *mechanism-agnosticism*: the property, established in Section 5, that the forecast reads only the query stream and is invariant to the noise mechanism and its sensitivity. We confirm it: on a workload that issues each logical query in one of several equivalent syntactic forms, the realized number of distinct equivalence classes matches $\mathbb{E}[u_k]$ over the class distribution to within 0.7% on average, whereas a byte-identical cache over-counts the distinct queries by 2.5–4.3× (the variant multiplicity) and would over-spend the budget by the same factor.

Proposition 1 (Expected unique queries). $\mathbb{E}[u_k] = \sum_{i=1}^{m} \left[1 - (1-p_i)^k\right]$.

Proof. Let $X_i = \mathbf{1}[\text{template } i \text{ appears}]$. Then $\mathbb{P}[X_i = 0] = (1-p_i)^k$, so $\mathbb{E}[X_i] = 1 - (1-p_i)^k$; linearity of expectation gives the result. □

$(1-p_i)^k$ is the chance query i is *never* asked across the k requests; $1-(1-p_i)^k$ is the chance it is asked *at least once*. Summing over all m possible queries gives the expected number of *distinct* queries that actually appear, which are the only ones that ever cost budget. As a

worked example, used throughout the paper, consider a dashboard with three tiles asked 70%/20%/10% of the time (p=(0.7, 0.2, 0.1)) and refreshed k=100 times: each tile is almost surely asked at least once ($1-0.3^{100}$, $1-0.8^{100}$, $1-0.9^{100}$, all ≈ 1), so $\mathbb{E}[u_k] \approx 3$. The workload therefore pays for about 3 distinct queries rather than 100 refreshes, and the formula establishes this *before* any query runs.

Proposition 2 (Budget consumption and savings). *Naive sequential composition spends $\varepsilon_{\text{naive}}(k) = k\varepsilon_q$. Under workload-aware exact-repeat accounting,*

$$\mathbb{E}[\varepsilon_{\text{wa}}(k)] = \varepsilon_q\,\mathbb{E}[u_k],$$
$$S(k) = 1 - \frac{\mathbb{E}[\varepsilon_{\text{wa}}(k)]}{\varepsilon_{\text{naive}}(k)} = 1 - \frac{1}{k}\sum_i \left[1-(1-p_i)^k\right].$$

Budget is charged only the *first* time each distinct query is seen, so the saving $S(k)$ is the fraction of requests that are exact repeats. In the three-tile example, $S(100) = 1 - 3/100 = 97\%$: ninety-seven of every hundred refreshes are free.

Three limiting cases serve as sanity checks by evaluating the formula at its corners. *(A) Deterministic repetition* (p_1=1): $S(k) = 1 - 1/k$—the folklore caching rule, now a corner of Proposition 2 at maximum skew, not a standalone claim. *(B) Uniform, $m \to \infty$*: workloads almost never repeat, $\mathbb{E}[u_k] \to k$, $S(k) \to 0$, recovering naive composition. *(C) Uniform, $k \to \infty$, m fixed*: every template eventually appears, so the budget saturates at $m\varepsilon_q$ regardless of k—the practical regime in which a finite-template workload cannot exhaust a budget large enough to cover all m templates.

Proposition 3 (Zipf-parameterized workload). *For $p_i \propto i^{-\alpha}$, the savings ratio $S(k)$ is monotone non-decreasing in α for fixed (k, m). As $\alpha \to \infty$ it approaches Limit A. As $\alpha \to 0$ it approaches the uniform case: naive composition (Limit B)* only when *$m \to \infty$; for finite m with $k \gtrsim m$ the budget instead saturates at $m\varepsilon_q$ (Limit C), which still yields substantial savings $S \approx 1 - m/k$.*

Proof sketch. $\mathbb{E}[u_k] = \sum_i \phi(p_i)$ with $\phi(x) = 1-(1-x)^k$ concave ($\phi'' \le 0$), so $\mathbb{E}[u_k]$ is Schur-concave. Larger α majorizes the weight vector (mass shifts onto top templates), hence lowers $\mathbb{E}[u_k]$ and raises $S(k)$; the endpoints follow from the uniform and point-mass cases. A numerical scan over $m \in \{3, \ldots, 100\}$, $k \in \{2, \ldots, 200\}$ found no violation. □

The Zipf(α) family is a standard parameterization of workload skew: α=0 makes every query equally likely, and larger α concentrates the requests on a few popular queries, producing a steeper head and a longer tail. The proposition states that the more skewed the workload, the more it repeats and the more budget it saves, which is the expected monotone trend. *Schur-concavity* and *majorization* are the formal tools that make the implication "more concentrated ⇒ fewer distinct queries" precise.

Proposition 4 (Concentration of u_k). *Writing $u_k = \sum_i X_i$ with $q_i = 1-(1-p_i)^k$, the occupancy indicators are negatively associated, so* $\text{Var}[u_k] \le \sum_i q_i(1-q_i) \le m/4 =: V$, *and Chebyshev gives $\mathbb{P}[|u_k - \mathbb{E}[u_k]| \ge t] \le V/t^2$.*

Concentration means that the budget *actually* spent on one particular run stays close to the forecast, rather than being correct only on average over many runs, so a custodian can rely on the single forecast number. Negative association of the indicators means that once one query has appeared, the others are, if anything, slightly *less* likely to also appear, which only tightens the spread around the mean. Because $V = O(m)$ rather than $O(k)$, the realized u_k has standard deviation $\sqrt{V} = O(\sqrt{m})$ (0.11–1.77 here at m=20, small relative to $\mathbb{E}[u_k] \approx 15$–20): the spread grows only as $\sqrt{m}$, not $\sqrt{k}$. This bounds the *scale* of single-run deviation—a Chebyshev statement, not a tight high-probability guarantee for a given tolerance (a distribution-free McDiarmid bound $2e^{-2t^2/k}$ also holds but is $\sim 18\times$ looser once u_k saturates, so we use it only as a fallback). The budget-exhaustion tail follows: for $c/\varepsilon_q > \mathbb{E}[u_k]$, $\mathbb{P}[u_k\varepsilon_q \ge c] \le V/(c/\varepsilon_q - \mathbb{E}[u_k])^2$.

Proposition 5 (Utility under a fixed total budget). *With a fixed total ε_{tot}, naive affords $\varepsilon_q^{\text{naive}} = \varepsilon_{\text{tot}}/k$ per query and workload-aware affords $\varepsilon_q^{\text{wa}} = \varepsilon_{\text{tot}}/\mathbb{E}[u_k]$ per unique release; since $\mathbb{E}[|\eta|] = \Delta f/\varepsilon_q$, the predicted per-query error ratio is $k/\mathbb{E}[u_k]$.*

With a fixed total budget, naive accounting must spread it thinly across all k requests, whereas workload-aware accounting spreads it across only the $\mathbb{E}[u_k]$ *distinct* ones—so each released answer receives a $k/\mathbb{E}[u_k]\times$ larger share of the budget and proportionally less noise. In the three-tile example (k=100, $\mathbb{E}[u_k]\approx$3) that is up to $\approx 33\times$ less error per answer at the *same* privacy cost, in expectation (see the caveat below on capping spend when the realized u_k exceeds $\mathbb{E}[u_k]$).

At Zipf α=1, m=10, k=100, $\mathbb{E}[u_k] \approx 9.9$, so workload-aware accounting answers the same workload at roughly $10\times$ lower per-query error at equal total budget. Two caveats are in order. First, the sizing $\varepsilon_q^{\text{wa}} = \varepsilon_{\text{tot}}/\mathbb{E}[u_k]$ targets the *expected* budget: since the realized u_k can exceed $\mathbb{E}[u_k]$, an allocator must cap per-query spend at the remaining budget (and reject once exhausted) to enforce a hard B—which is why the static-regime safe choice is $\varepsilon_q = B/m$ ($u_k \le m$, never overruns). Second, a cached answer reuses a *single* noise draw, forgoing the variance reduction of k independent draws. The budget saving and the forgone averaging are therefore two consequences of the same reuse decision.

3.3 System and temporal extension

The middleware is a thin proxy between the analyst and a backend database (Figure 2). Each query is parsed and validated, hashed to a (template, parameter) key, and checked against a two-level cache; a hit is returned for free (post-processing), a miss is charged ε_q, noised, and cached. Four core execution modes (exact, naive-DP, workload-aware-DP, temporal-DP) share the pipeline of Algorithm 1; two further modes extend it (predictive allocation, Appendix A; semantic caching, Appendix D). The pipeline couples cache validity to a temporal regime: it advances a logical clock, simulates per-step Bernoulli invalidations (each entry invalidated independently with probability λq_{inv}, a discrete approximation of a Poisson process), ages each entry against the staleness tolerance τ, and only then returns the cached release or charges the ledger.

A *temporal extension* couples budget to time, since in deployment the two cannot be separated: data updates invalidate cached answers and freshness requirements force re-noising. This is a *first-order* treatment, a planning estimate rather than a full model. A complete staleness/update theory (with re-release policies and a

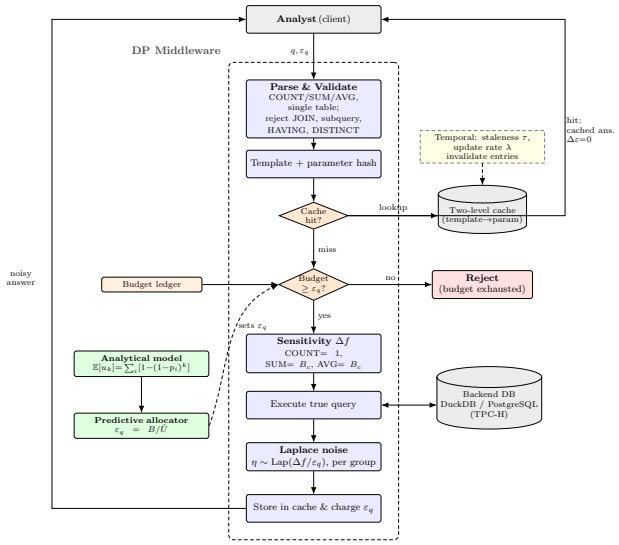

Figure 2: Workload-aware DP-SQL middleware. A cache hit is free by post-processing; a miss is charged ε_q, noised, and cached. The analytical model drives the predictive allocator (ε_q=$B/\hat{U}$), and a temporal regime (τ, λ) invalidates stale entries.

Algorithm 1 Workload-aware DP middleware (one query).

Require: Query q, budget ε_q, ledger $\mathcal{L}$, staleness τ, update $(\lambda, q_{\text{inv}})$

Parse/validate q; advance clock; invalidate each entry w.p. λq_{inv}
Compute template hash h_T, parameter hash h_P
if $\mathcal{L}$.cache$[h_T][h_P]$ fresh (age $\leq \tau$) **then**
  return cached $\tilde{r}$ (post-processing, $\Delta\varepsilon$=0)
end if
if $\mathcal{L}$.remaining $< \varepsilon_q$ **then**
  return BudgetExhausted
end if
$\Delta f \leftarrow$ sensitivity; $\mathcal{L}$.consumed += ε_q
$r \leftarrow$ execute q; $\tilde{r} \leftarrow r + \mathrm{Lap}(\Delta f/\varepsilon_q)$ per aggregate, per group
Cache and **return** $\tilde{r}$

proper arrival process) is a study in its own right, which we leave to future work.

Within this first-order treatment we work with a temporal *planning estimate*, which is neither a proven bound nor an equality: if every template were re-queried in every freshness window, the number of (re-)noisings per template would be $N = \max(1, \lceil T/\tau \rceil) + T\lambda q$, giving the first-order estimate $\mathbb{E}[\varepsilon_{\text{temp}}] \approx \varepsilon_q \mathbb{E}[u_k]\, N$. Because N assumes every template is re-issued in *every* window—ignoring the actual arrival schedule—the estimate is conservative: across independently seeded trials it over-predicts the realized budget in every cell with active dynamics (at τ=10 it predicts ≈70 vs. a realized ≈38; at λ=0.2, ≈49 vs. ≈32; Section 4), so we commit to the *trend*. Under a *specified* arrival model, moreover, this estimate is a proven *upper* bound:

Proposition 6 (Temporal budget upper bound). *If the query-arrival process is independent of the data-update process, updates arrive as a Poisson(λ) process and invalidate each cache entry independently with probability q, and a stale (*age $> \tau$*) or invalidated entry is re-noised on its next query, then $\mathbb{E}[\varepsilon_{\text{temp}}(T)] \leq \varepsilon_q \mathbb{E}[u_k]\, N$ with $N = \lceil T/\tau \rceil + \lambda T q$.*

Proof. Per template, partition re-noisings into staleness- and update-driven. A staleness re-noising resets the entry's age, so at most $\lceil T/\tau \rceil$ occur. Each update-driven re-noising consumes a distinct invalidation event, so their number is at most the invalidations hitting the template, whose expectation is $\leq \lambda T q$. Summing over the (independent) appeared templates gives $\mathbb{E}[u_k]\, N$; multiply by ε_q. □

This holds for the Poisson model above, *not* for every arrival process; we validate it—the bound holds in 12/12 simulated cells, strict and conservative (realized/bound 0.15–0.57) in the dynamic regime and tight (equality) in the static one. The only *unconditional* hard ceiling remains naive composition, $k\varepsilon_q$. Three regimes slide continuously: *(a) static* ($\tau \to \infty, \lambda$=0, N=1) recovers Section 3.2; *(b) bounded staleness* (τ finite, λ=0); *(c) update-driven* (λ>0). On the implementation side, the prototype is unit-tested across parser validation, sensitivity, Laplace calibration, cache semantics, the model limits, the concentration bound, and temporal re-noising.

While the *magnitude* estimate above is not an unconditional bound, the re-release *policy* that consumes it admits one. The planning estimate prices a schedule; the question a custodian actually faces is whether re-noising can overrun B. It cannot, if releases are budget-gated:

Proposition 7 (Budget-aware re-release: a provable ceiling). *Fix a per-release cost ε_q and total budget B, and consider the policy that, at each freshness tick, re-noises a stale or update-invalidated template only while the remaining budget is $\geq \varepsilon_q$, and otherwise serves the last cached answer. Then for* any *freshness interval τ and* any *update process (volatility λ), the cumulative spend never exceeds B, and the number of re-releases is exactly $\min\big(R, \lfloor B/\varepsilon_q \rfloor\big)$, where R is the number of refreshes the schedule requests.*

Proof. Each release decrements the remaining budget by ε_q and is issued only when the remaining budget is $\geq \varepsilon_q$, so the remaining budget stays ≥ 0 and the cumulative spend is non-decreasing and bounded above by B. Once $\lfloor B/\varepsilon_q \rfloor$ releases have been made the remaining budget is $< \varepsilon_q$ and every further refresh is denied, giving the stated count. □

This is a worst-case *guarantee*: it holds for every arrival process, unlike the magnitude estimate, and it makes the refresh rate a budgeting decision rather than an unverified assumption. The guarantee also turns the refresh rate into a formal *optimization* problem in the remaining budget *and* the volatility λ. Consider minimizing the time-average staleness, which is proportional to the refresh interval τ, subject to the re-release budget constraint $\varepsilon_q\, u_k \big(\lceil T/\tau \rceil + \lambda T q\big) \leq B$ (the $\lambda T q$ term is the volatility-driven spend of Proposition 6). The

Table 2: Model vs. empirical u_k, Zipf (m=7, 30 trials/cell). Parentheses: signed relative error (%).

$\alpha \backslash k$	10	25	50	100
0.0	5.50/5.53 (−0.6)	6.85/6.77 (1.2)	7.00/7.00 (0.0)	7.00/7.00 (0.0)
0.5	5.30/5.23 (1.3)	6.73/6.73 (−0.1)	6.98/6.93 (0.7)	7.00/7.00 (0.0)
1.0	4.73/4.70 (0.6)	6.32/6.30 (0.3)	6.88/6.90 (−0.3)	7.00/7.00 (0.0)
1.5	3.98/3.87 (2.8)	5.57/5.70 (−2.3)	6.48/6.40 (1.3)	6.91/6.93 (−0.3)
2.0	3.25/3.13 (3.5)	4.64/4.70 (−1.3)	5.69/5.73 (−0.7)	6.50/6.43 (1.1)
3.0	2.19/2.07 (5.7)	3.07/3.33 (−8.6)	3.85/3.90 (−1.3)	4.72/4.67 (1.1)

objective is monotone increasing in τ while the feasible set is monotone in τ, so the optimum sits on the budget boundary:

$$\tau^\star(\lambda) = \frac{T}{B/(\varepsilon_q u_k) - \lambda T q}, \qquad \text{valid when } \lambda T q < \tfrac{B}{\varepsilon_q u_k}.$$

This is the budget- and volatility-aware optimal refresh: a shorter τ violates the budget (trips the gate, leaving late ticks stale), a longer one wastes affordable freshness, and a higher update rate λ *raises* $\tau^\star$ because updates already consume part of B. At λ=0 it recovers $\tau^\star = T u_k \varepsilon_q / B$; once $\lambda T q \geq B/(\varepsilon_q u_k)$ the update load alone exhausts the budget and no staleness-driven refresh is affordable (we verify $\tau^\star(\lambda)$ spends exactly B at the boundary). The remaining open piece is the full accuracy theory of $\mathbb{E}[\varepsilon_{\text{temp}}]$ under a general renewal arrival process.

4 Results and Findings

This section reports the empirical findings. The single most important result is the real-trace forecast of Section 4.2: predicting a workload's full distinct-query count, and hence its budget, from only its first half cuts the error by 45% over the plug-in estimate. The synthetic campaign below first establishes the model's arithmetic and invariances before turning to that real-trace evidence.

The experimental campaign comprises six sweeps totaling 4,155 trials (~150,000 queries) over Zipf workloads (m=7 Adult age-bands and TPC-H templates), four per-query budgets $\varepsilon_q \in \{0.1, 0.5, 1, 2\}$, lengths $k \in \{10, \ldots, 500\}$, and two data scales (TPC-H at scale factor (SF) 1 with 6M lineitem rows, and SF=10 with 60M). We compare internal execution modes under one identical middleware so that the algorithmic effect of workload-aware accounting is isolated; an end-to-end comparison against external prototypes would require query-rewriting integration, which we leave as future work.

4.1 Validating the model

The first validation checks the model's $\mathbb{E}[u_k]$ against measurement: Table 2 compares the closed form to the 30-trial empirical mean across the main grid, and agreement is within < 3% in 21 of 24 cells; the three outliers (3.5–8.6%) occur at high skew and small k, where $\mathbb{E}[u_k]$ is tiny and a 0.1-unit deviation is a large relative error. The savings ratio $S(k)$, computed from measured budget, matches within < 2% in 23/24 cells.

Beyond pointwise agreement, the predicted invariances and limiting behavior also hold empirically. The model's $\mathbb{E}[u_k]$ is structurally independent of ε_q (confirmed within 0.12 units across $\varepsilon_q \in \{0.1, 0.5, 1, 2\}$) and of data scale: the same templates over a 10× larger TPC-H database (60M rows) give identical empirical $\mathbb{E}[u_k]$ to within 0.034 units, isolating workload structure from data scale. Pushing skew to α=10 at k=200, predicted and empirical $\mathbb{E}[u_k]$ stay within 0.13 units while $\mathbb{E}[u_k]$ itself varies by an order of magnitude, and $S(k) \to 99.4\%$ recovers the $1 - 1/k$ corner numerically (Figure 3).

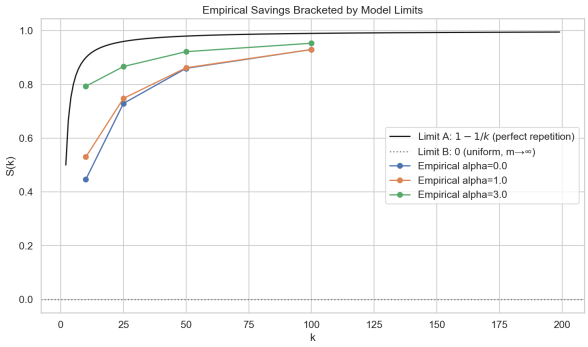

Figure 3: Savings ratio $S(k)$ vs. workload length, several Zipf α. Dashed: model. Markers: empirical mean (30 trials). High-α curves approach $1 - 1/k$ (Limit A); the α=0 curve stays near 0 until k approaches m, then saturates (Limit C, finite m).

The validation so far assumes i.i.d. arrivals. To probe the forecast's robustness to this assumption, we extend the occupancy model to correlated and non-stationary arrival processes, beginning with a bursty "sticky" process.

Proposition 8 (Sticky-Markov occupancy). *Under the sticky arrival process—$X_1 \sim \{p_i\}$, and for $t \geq 2$, $X_t = X_{t-1}$ with probability s else a fresh draw $\sim \{p_i\}$ (a bursty workload with stationary marginal $\{p_i\}$)—the expected distinct count is*

$$\mathbb{E}[u_k] = \sum_i \Big[1 - (1-p_i)\big(s + (1-s)(1-p_i)\big)^{k-1}\Big].$$

Proof. The set of distinct templates visited equals the set of distinct values among the *fresh* draws (a repeat only re-emits the current template), and the number of fresh draws is $F = 1 + \text{Binom}(k-1, 1-s)$. Conditioning on F and using $\mathbb{E}[z^{\text{Binom}(n,1-s)}] = (s + (1-s)z)^n$ with $z = 1 - p_i$ gives the result. At s=0 it recovers the i.i.d. $\mathbb{E}[u_k]$; at s=1 it gives 1. □

Proposition 9 (General-Markov occupancy). *For an* arbitrary *Markov arrival process with transition matrix P ($P[j,l] = \Pr[X_{t+1}=l \mid X_t=j]$) and initial law ν, let Q_i be the sub-stochastic block of P on the states $\neq i$ and ν_i the initial mass on those states. Then $\mathbb{E}[u_k] = \sum_i \big[1 - \nu_i^\top Q_i^{k-1} \mathbf{1}\big]$.*

Proof. State i is unvisited iff the chain stays in $\{\cdot \neq i\}$ for all k steps, an event of probability $\nu_i^\top Q_i^{k-1}\mathbf{1}$; summing the complements over i gives the result. The i.i.d. form (every row of P equal to $\{p_i\}$) and the sticky form of Proposition 8 are special cases. □

This closes the occupancy model for *any* Markovian arrival pattern; it reproduces both special cases to machine precision and matches simulation to $\leq 10^{-3}$ on random chains.

Proposition 10 (Latent-state (Markov-modulated) occupancy). *When the query distribution is itself non-stationary—a hidden Markov chain with transition matrix T and initial law ω selects, in state h, an emission distribution $p^{(h)}$ over the templates—the expected distinct count is*

$$\mathbb{E}[u_k] = \sum_i \Big[1 - \omega^\top D_i \,(TD_i)^{k-1}\mathbf{1}\Big], \qquad D_i = \mathrm{diag}\big(1 - p_i^{(h)}\big),$$

where D_i collects the per-state probabilities of not *emitting template i.*

Proof. Template i is unvisited iff every step avoids it; weighting the hidden paths by their probability of avoiding i gives $\omega^\top D_i(TD_i)^{k-1}\mathbf{1}$, and summing the complements over i gives $\mathbb{E}[u_k]$. A single hidden state recovers the i.i.d. form, and a deterministic emission ($p^{(h)}{=}e_h$) recovers Proposition 9, and through it the sticky and i.i.d. cases. □

This covers regime-switching workloads whose $\{p_i\}$ drifts with a latent state. On a two-regime chain, each regime concentrating on a different half of the templates, the closed form matches simulation to ≤ 0.03, whereas a stationary i.i.d. forecast that ignores the regimes mis-predicts u_k by up to 1.7; the latent structure must therefore be modeled explicitly, which the proposition provides.

Across these robustness extensions, four properties make the forecast dependable. *(i) Distribution-agnostic*: it is exact for any i.i.d. marginal (uniform, Zipf, lognormal-shaped), matching the empirical mean to ≤ 0.2. *(ii) Concentrated*: by Proposition 4 the standard deviation is small, $\sqrt{V}$=0.11–1.77 at m=20, so the spread around the forecast is tight. This is a variance- or Chebyshev-scale statement, not a high-probability envelope. *(iii) Exact on heterogeneous workloads*: on a constructed mixed W1+W4 workload with true $u_k = 1 + (1-f)k$, the closed form $\varepsilon_q u_k$ matches the measured budget exactly at every mixing fraction (Table 3). *(iv) Conservative under* positive *correlation*: a bursty sticky-Markov process (the tested case; same stationary marginal) clusters repeats and lowers the distinct count, so the i.i.d. forecast *over*-predicts u_k by 1.09/1.29/2.72× at stickiness s=0.3/0.6/0.9—the safe direction. This over-prediction is now *exact*: Proposition 8 gives the sticky-process $\mathbb{E}[u_k]$ in closed form (matching simulation to < 0.08 across 16 cells)—extending the occupancy model from i.i.d. to a Markovian arrival process—and the i.i.d. forecast is its s=0 special case, exceeding it for s>0, so the burstiness gap is quantified exactly rather than merely bounded by $u_k \leq m$. The conservative-direction guarantee holds only for this positive-correlation process. Anti-correlated or without-replacement arrivals would instead push u_k above i.i.d. and make the forecast optimistic. The unconditional safety net is therefore the $\varepsilon_q{=}B/m$ allocator: in the *static* regime $u_k \leq m$ for any arrival process, so it never rejects (under temporal re-noising the same query can be charged again, so this becomes a per-window guarantee).

The temporal extension of Section 3.3 is validated in the same manner: we sweep τ (with λ=0) and λ (with τ large), over independently seeded trials so each draws an independent update stream. The planning estimate $\varepsilon_q\mathbb{E}[u_k]N$ tracks the empirical mean in trend and matches at large τ (rare re-noising); in every cell with active dynamics it *over*-predicts the realized budget (at τ=10, ≈70 vs. an empirical ≈38; at λ=0.2, ≈49 vs. ≈32), since not all re-noisings materialize within the horizon. Under the Poisson update model of Proposition 6 it is moreover a proven *upper* bound, which we confirm holds in 12/12 simulated cells; for an unspecified arrival process we report it as a conservative planning estimate. In both regimes, freshness carries an explicit, bounded cost.

Table 3: Mixed W1+W4 workload, B=50, ε_q=0.5, 20 trials. The closed form matches the measured budget exactly.

f	true u_k	measured ε used	predicted $\varepsilon_q u_k$
0.1	91	45.5	45.5
0.3	71	35.5	35.5
0.5	51	25.5	25.5
0.7	31	15.5	15.5
0.9	11	5.5	5.5

4.2 Real-workload validation (Redbench)

The synthetic sweep cannot establish *external* validity, so we add the first test on production-derived traces: Redbench [23], 30 analytical SQL workloads synthesized from Amazon Redshift's Redset query log, in which each user's real query timeline (and thus its repetition structure) is mapped onto CEB+/JOB query instances. Redbench is *not* a differential-privacy benchmark; we use its production-derived query-repetition structure solely to externally validate our workload-occupancy forecast, not any DP mechanism of its own. The exact-query identity is the SQL instance, so Redbench's own *query-repetition rate* $= 1 - u_k/k$, computed from the realized distinct count u_k, is exactly our $S(k)$. Running the occupancy model against all 30 workloads yields five findings. *(1) Real workloads span the full skew range:* realized $S(k)$ runs from 0% to 98% (mean 53%), so template repetition (the structure the model exploits) is pervasive in production rather than an artifact of the sweep. *(2) Real distributions are heavy-tailed:* fitting Zipf to each workload gives $\alpha \in [0, 3]$ with mean 0.84 (near classic Zipf), and our synthetic sweep $\alpha \in \{0, \ldots, 3\}$ *brackets* the real range, which grounds the Zipf parameterization of the synthetic sweep in production data. *(3) The descriptive fit is tight:* when fed the full-trace $\{p_i\}$, the closed form reproduces the realized savings to a mean absolute 0.13 in $S(k)$—but this is an *in-sample, descriptive* check (it uses the whole trace), not a before-execution forecast. *(4) The before-execution forecast succeeds* (our headline real-data result): estimating $\{p_i\}$ from only the *first* 50% of each timeline and predicting the full u_k, the smoothed Good–Toulmin estimator [20] cuts the plug-in's error by 45% (0.22 → 0.12 in u_k/k). The *plug-in* simply assumes the rest of the workload repeats the query mix seen so far, whereas the *Good–Toulmin* (unseen-species) estimator also predicts how many *brand-new* queries are still to come, the classic "how many species have we not yet seen?" problem, so it forecasts u_k more accurately from a short prefix. This is the genuine forecast rather than the in-sample fit, and on production-derived data it confirms that unseen-species correction is the right tool. Redbench is trace-*derived* synthesis (real repetition structure, CEB+/JOB queries, not raw production SQL), so it raises external validity substantially without fully eliminating the gap (Section 7). *(5) The forecast survives a controlled out-of-distribution check:* to rule out that it is an

artifact of the Zipf self-consistency setup, we run the *same* prefix-forecaster on synthetic workloads drawn from five families it is never told about—two Zipf, plus uniform, lognormal-shaped, and a heavy Pareto tail. Forecasting the full u_k from a 50% prefix, the smoothed Good–Toulmin estimator stays within a mean 0.021 (max 0.026) in u_k/k across *all five*, Zipf and non-Zipf alike, cutting the naive prefix-distinct error by 76%: the forecast is not tied to the Zipf assumption. Beyond horizon $t{=}(k{-}n)/n \approx 1$ (e.g. a 25% prefix) the unseen-species series degrades, as is well known for Good–Toulmin, so we clamp the estimate to $[D_n, k]$ (a distinct count cannot exceed k), which turns any divergence into the budget-safe over-forecast.

4.3 Budget savings and where naive composition fails

The budget savings themselves follow the model's predictions. Across the full grid of six workloads, workload-aware accounting answers the same 100 queries at comparable per-query MAE while spending only $\varepsilon_q u_k$: savings of 100× on the repetitive W1, then 33×, 20×, and 14.3× on the TPC-H and uniform pools (where u_k saturates at the pool size m), and *exactly* 1× on the genuine drill-down W4, where every query is unique and the model correctly predicts $S(k)$=0 (Table 4, Figure 4). These multipliers are accounting ratios $1/(1-S(k))$ that any exact-repeat cache attains; our contribution is the closed-form *prediction* of $S(k)$ before execution (within < 3%), not the saving itself.

Table 4: Full grid (6 workloads, k=100, ε_q=1, 30 trials). Workload-aware spends $\varepsilon_q u_k$ at 93–99% cache-hit rates; the drill-down (W4) is the predicted no-win case.

Workload	naive ε	work.-aware ε	savings
W1 Repetitive	100.0	1.0	100×
W2 TPC-H returnflag	100.0	3.0	33×
W2 TPC-H priority	100.0	5.0	20×
W2 Zipf α=1	100.0	7.0	14.3×
W3 Uniform	100.0	7.0	14.3×
W4 Drill-down (unique)	100.0	100.0	1×

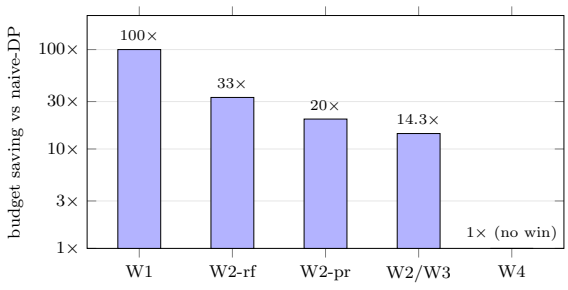

Figure 4: Budget saved vs. naive (k=100, ε_q=1, log scale): up to 100× on the repetitive W1 and 14–33× on the parametric/uniform pools, but exactly 1× (no win) on the unique drill-down W4—the model predicts the $S(k)$=0 case rather than overclaiming.

Two concrete failure regimes of naive composition sharpen this comparison. On the real middleware (Adult, B=10, k=100, 12 trials; Table 5), repetition turns the structural advantage into a stark gap. *(A) Budget exhaustion*: a dashboard re-issuing one tile at fixed ε_q=1 exhausts naive after B/ε_q=10 queries (10/100 answered), while caching answers all 100/100 for free—a 10× gap. *(B) Accuracy at fixed total budget*: to answer all 100, naive must thin the budget ($\varepsilon_q{=}B/k{=}0.1$, error ≈ 10), whereas concentrating it on the u_k=1 distinct release makes the answer almost exact. *(C) Control*: on a genuine drill-down both modes match at error ≈ 9.6; we include this case because the model predicts it to be the no-win case.

Table 5: Naive's failure regimes vs. workload-aware (real middleware, Adult, B=10, k=100).

Workload	Naive ans.	Naive |err|	Workload-aware ans.	Workload-aware |err|
Repetitive, fixed ε_q=1	10/100	0.9	**100/100**	0.4
Repetitive, fixed budget B	100/100	10.0	100/100	≈ **0**
Drill-down (all unique)	100/100	9.6	100/100	9.6

4.4 Predictive allocation

The validated model is not only descriptive but also *prescriptive*: an allocator estimates $\hat{U} = \mathbb{E}[u_k]$ online (after a short warmup) and sets $\varepsilon_q = B/\hat{U}$ to target the offline optimum $\varepsilon_q^* = B/u_k$ *under a uniform per-distinct-release objective* (a frequency-/importance-weighted error tilts ε toward the frequent templates—Appendix C), capping spend at B. At a fixed *total* budget this trades coverage for accuracy: on Zipf workloads (k=100, B=10) it lowers per-query MAE on the *answered* subset (Table 6; it answers 69–88/100 where naive answers 100) by 16.8% (α=0) down to 5.4% (α=2). The gain is only *borderline* (paired t-test $p \approx 0.05$ at α=0, not crossing 0.05; $p > 0.3$ for $\alpha \geq 1$); the improvement is directional rather than uniform, and it is bought with late-query rejections. The robust result is instead the *safe* allocator $\varepsilon_q{=}B/m$, which never rejects in the static regime ($u_k \leq m$; temporal re-noising can charge the same query again) and matches a learned bandit's best operating point at zero exploration cost, as the next comparison details.

A practical comparison places the safe allocator against a learned baseline. Against an ε-greedy learned bandit that must *explore* to find a good per-query ε, the closed-form safe allocator $\varepsilon_q{=}B/m$ reaches the bandit's best operating point at *zero* exploration cost: on the Zipf allocation benchmark it attains per-release MAE 2.0 versus the bandit's 3.6 at the same answer rate, because the forecast hands the allocator $\hat{U}$ directly instead of learning it online through trial releases. The bandit's own arm set includes the B/m denominator, so the closed form supplies, without exploration, the operating point the bandit must otherwise pay to find. This result shows the closed form reaching the bandit's operating point at zero cost rather than beating it, and the safe allocator's advantage narrows for $m{>}k/4$ where B/m is no longer the best arm. The budget sweep (32% savings at B=1, growing with B) and scale-invariance (40% at SF= 10) show the allocator is not tuned to a single operating point. The estimate $\hat{U}$ is produced by an online *workload-learning* step—a Laplace-smoothed empirical $\{p_i\}$ refined by the smoothed Good–Toulmin unseen-species correction (Section 4.2)—which already

generalizes across distribution families (finding 5) and over real Redbench traces. The full head-to-head, the learned-bandit curves, and the predictive ablation are in Appendix A.

The forecast must also remain accurate under workload drift: the $\{p_i\}$ a deployment learns online need not be stationary, and a system that re-estimates it should track change. We therefore add a drift-aware streaming learner, an exponentially weighted moving average (EWMA) that decays stale template mass and re-centers $\hat{U}$ on the current regime. On a workload whose distribution *shifts* mid-stream it tracks the post-shift horizon about 15× more accurately than a non-forgetting full-history plug-in. It is a planning-layer estimator, so privacy still rests on the public cap m and the ledger; the latent-state occupancy of Proposition 10 is the matching closed-form forecast for such regime-switching streams.

Table 6: Predictive allocator vs. fixed-ε_q baselines (mean ± standard error (SE), 30 trials, k=100, B=10). MAE is over answered queries; the allocator answers fewer (69–88) than naive (100). The gain is borderline ($p \approx 0.05$) only at α=0.

α	Mode	MAE	answered	ε spent
0.0	Naive	9.86	100/100	10.0
	Predictive	**8.21**±0.85	69/100	10.0
1.0	Naive	9.81	100/100	10.0
	Predictive	**8.98**±0.93	78/100	10.0
2.0	Naive	9.96	100/100	10.0
	Predictive	**9.42**±1.40	88/100	10.0

To place these results against prior systems, we frame the comparison as concrete situations a custodian faces; on the raw savings numbers we are level with prior work rather than ahead of it. Before any query executes, the forecast supplies a t=0 completion check (a hard yes under B/m, since $u_k \le m$) and a *per-release* (α, β) accuracy verdict (Appendix B), plus capacity planning from a prefix (the 45% Redbench result); reactive systems surface these only by spending budget query-by-query. On the savings *number* itself we do not lead: we cite the baselines' own published figures rather than re-running them (Turbo 1.7–15.9× [13], CacheDP a reduction under an accuracy contract [14]) against our 14–33× on the pools and 100× on pure repetition: the same order of magnitude, so we claim *parity*, not superiority (the workloads and mechanisms differ, so this is a ballpark placement, not a controlled run). A controlled, same-hardware comparison was not feasible within this study: the implementation of the academic PrivateSQL prototype was never publicly released (the paper, of course, is public) and Chorus is no longer maintained, while Turbo and the CacheDP research artifact are available but expose linear-query interfaces to which our SQL workloads would first have to be ported—the concrete head-to-head we identify in Section 6. At a fixed budget the safe ε_q=B/m allocator simply *reaches* an ε-greedy bandit's best operating point at zero exploration cost, as shown above (Appendix A). The contribution is the *before-execution* deliverable the others lack, not a larger multiplier.

4.5 Privacy leakage

We next verify that the accounting does not weaken the privacy guarantee itself: two attacks confirm the per-query calibration. *Membership inference*: an adversary observing one noisy COUNT must decide whether a target is present. The optimal single-release attacker's *accuracy* is bounded by $e^\varepsilon/(1+e^\varepsilon)$; the empirical area under the ROC curve (AUC)—which for this symmetric single-release threshold test nearly equals the attacker's accuracy—stays at or below that bound (within ≤ 1%) and is at chance for $\varepsilon \le 0.1$ (Table 7, left). A fully type-matched comparison would use the exact Laplace AUC; the two nearly coincide here. *Reconstruction*: replaying a 5-level differencing drill-down on real Adult counts (48842, 34327, 24137, 7210, 3137), the mean absolute error scales as $1.5/\varepsilon$, the expected absolute difference of two $\mathrm{Lap}(1/\varepsilon)$ variables, to within a few percent across three orders of magnitude (Table 7, right, indexed by the *per-release* ε). The attack is *five* releases, so by sequential composition it costs a *cumulative* 5ε; at a fixed total budget B the per-release value is ε=$B/5$ and the error scales accordingly: the Dinur–Nissim view ties reconstruction to the total budget spent, not the per-query value. (A stronger shadow-model membership inference attack (MIA) at our sample size was inconclusive, a negative result that we report in the artifact rather than as a leakage claim.) Our calibration conclusion rests on the clean single-query MIA and the reconstruction above, both of which degrade exactly where the model predicts efficient budgeting (high skew, low ε_q); the reconstruction in particular succeeds on the W4 drill-down, which the model flags as the no-savings case.

Table 7: Leakage vs. per-release ε. Left: single-release membership-inference AUC vs. the DP *accuracy* bound $e^\varepsilon/(1+e^\varepsilon)$ (AUC≈accuracy for this symmetric test). Right: differencing reconstruction error vs. the $1.5/\varepsilon$ reference (a 5-release attack; cumulative cost 5ε).

ε	AUC	$\frac{e^\varepsilon}{1+e^\varepsilon}$	ε	rec. err	$1.5/\varepsilon$
0.01	0.499	0.503	0.01	148.2	150.0
0.10	0.522	0.525	0.10	14.8	15.0
1.00	0.724	0.731	1.00	1.5	1.5
5.00	0.988	0.993	5.00	0.3	0.3

4.6 Runtime and overhead

Finally, we quantify the runtime overhead: the privacy machinery is cheap relative to I/O. Across 1,500 instrumented queries, cache hits are 2.7× faster than misses (0.76 vs. 2.02 ms) because they skip the database round-trip (36% of miss cost); SQL parsing is 21%, and the privacy-specific stages (sensitivity, allocation, noise) sum to under 0.25 ms. Tail latency is controlled (miss p99=5.1 ms, hit p99=3.6 ms), and single-threaded throughput is ∼495 q/s on the miss path and ∼1,320 q/s on the hit path, so a workload-aware mode running at a 93–99% hit rate sustains analyst-tier load. The asymptotic cost of the forecast itself is $O(m)$ per evaluation of $\mathbb{E}[u_k]$. A separate end-to-end measurement against raw DuckDB on *identical* SQL quantifies the practical-system overhead the deployed comparison demands: the full DP miss path adds roughly 2–6× over a bare query (sub-millisecond to ∼1.2 ms; hardware-dependent), an ε=0 cache hit is ∼5× faster (∼0.2–0.3 ms) and free in budget, and a mixed repeated workload sustains ∼1.4–1.6k queries/s at a ∼74% hit rate; the workload-aware DP middleware is therefore practical as a deployed layer.

In interpreting these results, the two validations should be weighed differently: the synthetic grid (the $< 3\%$ agreement) is a *self-consistency* check—it confirms the estimator's arithmetic, not that real workloads are i.i.d. Zipf—whereas the genuine external evidence is the Redbench prefix forecast, a 45% error reduction on traces the model never saw. Between the two sits the controlled out-of-distribution test (Section 4.2, finding 5): the deployable prefix-forecaster predicts u_k within $\approx 2\%$ on uniform, lognormal, and Pareto workloads it was never told the family of, so the headline accuracy is not an artifact of drawing the workload from the same Zipf the closed form integrates.

5 Beyond Single-Table Aggregates: Multi-Table Joins

A natural objection is that our closed-form models were derived for single-table COUNT/SUM/AVG, where the sensitivity Δf is a public constant (1 for COUNT, the column bound B_c for SUM/AVG; Section 3.1), and that they break for multi-table JOINs, whose sensitivity analysis is notoriously harder. This section separates the part that transfers unchanged from the part that genuinely degrades, and *measures* the transfer on a real TPC-H join (Figure 5).

Proposition 11 (Mechanism-agnostic forecast). *The expected distinct count $\mathbb{E}[u_k]$, the savings $S(k) = 1 - \mathbb{E}[u_k]/k$, the workload-aware spend $\varepsilon_q \mathbb{E}[u_k]$, and the relative utility ratio $k/\mathbb{E}[u_k]$ are functions of $(m, \{p_i\}, k)$ alone; they are invariant to the per-query DP mechanism and to its sensitivity Δf, and therefore transfer from single-table aggregates to any query class—joins included—unchanged.*

Proof. Each is built from $\mathbb{E}[u_k] = \sum_i [1 - (1 - p_i)^k]$ (Prop. 1), a function of the arrival distribution only; Δf enters solely the noise scale $\mathrm{Lap}(\Delta f/\varepsilon_q)$ and cancels in the ratio $k/\mathbb{E}[u_k]$. □

The single-table results of this paper are then the $d_{\max}$=1 instance of the join-parameterized framework below.

What transfers is the entire forecasting layer, because it reads the *query stream* alone: $\mathbb{E}[u_k]$ (Prop. 1), the savings $S(k)$ (Prop. 2), the limiting behavior, the skew-monotonicity (Prop. 3), the concentration bound (Prop. 4), and the spend $\varepsilon_q \mathbb{E}[u_k]$ depend on $(\{p_i\}, k, m)$ alone, never on what a query computes or how sensitive it is. Three quantities therefore transfer to a JOIN workload *unchanged*. The *spend* forecast holds because an exact JOIN repeat is still served free as post-processing; the *relative utility* ratio $k/\mathbb{E}[u_k]$ holds because Δf cancels, so it stays before-execution even when Δf is unknown; and *budget completion* holds because the $\varepsilon_q = B/m$ ledger never overruns, since $u_k \le m$ is purely combinatorial. What does *not* transfer for free is *answerability*. A general equi-join has unbounded global sensitivity, so a usable ε_q-DP answer needs a finite per-template sensitivity bound, and the *absolute* per-release (α, β) verdict, which reads Δf directly, inherits whatever that bound costs.

The one input that does change when moving to joins is the noise calibration, the scale $\mathrm{Lap}(\Delta f/\varepsilon_q)$: adding one tuple to R in $R \bowtie S$ can match arbitrarily many tuples in S, so the worst-case global sensitivity is unbounded and textbook Laplace calibration is undefined. The literature replaces the global constant with an instance-dependent or relaxed bound: smooth sensitivity [28], elastic sensitivity (FLEX), an efficiently computable bound from per-attribute max-frequencies [27], residual sensitivity for multi-way self-join-free counts [25], and R2T, which truncates each entity's contribution at a privately chosen threshold and is the first to handle foreign keys (FK) with self-joins [26]. None of these is a drop-in replacement: the smooth-sensitivity route generally costs an (ε, δ) relaxation, and R2T spends budget to pick τ and injects clipping bias. The conservative public route we adopt is a declared foreign-key multiplicity $d_{\max}$ (TPC-H caps line-items-per-order at 7, so $d_{\max}$=7): capping makes $\Delta f = d_{\max}$ a public global-sensitivity constant, the B_c-style bound we already use for SUM/AVG and looser than the instance-optimal options. The join COUNT is then calibrated with $\mathrm{Lap}(d_{\max}/\varepsilon_q)$, and the occupancy/spend forecast and B/m sizing apply unchanged. The cache, the distinct-release count, and the $O(m)$ forecast cost are untouched; only the ledger semantics (pure ε vs. (ε, δ)) and the absolute accuracy verdict are. Self-joins are the genuinely hard regime, since one entity can raise many others' degrees, which is exactly what R2T targets.

We confirm the preceding analysis empirically on TPC-H by running the worked example as a *twin* experiment: one Zipf identity stream over the m=5 order-priority pool, instantiated both as the single-table COUNT (Δf=1) and as the `orders`⋈`lineitem` COUNT ($\Delta f = d_{\max}$=7) on the real 1.5M-order / 6M-line-item tables (600 trials, $\alpha \in \{0, \ldots, 2\}$, $k \in \{10, \ldots, 100\}$). Confirming Proposition 11, the forecast is *identical* on the two sides: the realized distinct count, $\mathbb{E}[u_k]$, the savings $S(k)$, and the spend $\varepsilon_q u_k$ coincide trial-for-trial, since occupancy reads only the identity stream. It tracks the closed form within 1.4% on average (17/20 cells under 3%, the same accuracy as the single-table grid; Figure 5, left). The *only* quantity that moves is the per-release noise: the answered-release MAE is 7.6× larger on the join side (Figure 5, right), matching the $d_{\max}$=7 sensitivity ratio (expected exactly 7; the small excess is finite-sample mean-absolute noise over the few answered releases), while the relative utility ratio $k/\mathbb{E}[u_k]$ is unchanged because Δf cancels. The sensitivity-agnostic transfer is thus measured rather than merely argued: the closed-form forecast holds on a real multi-table join exactly as on the single table, and the join's added difficulty is confined to the orthogonal per-release Δf.

Beyond the offline twin experiment, the middleware also *executes* the FK join end-to-end under DP, so the transfer is a property of the running system rather than only of the analysis. A single two-table inner equi-join `COUNT` parses, is charged the public FK-multiplicity sensitivity $\Delta f = d_{\max}$ (the privacy unit is the parent entity, so removing one order removes up to $d_{\max}$ joined rows), is served the conservative $\mathrm{Lap}(d_{\max}/\varepsilon_q)$ noise, and has its exact repeats hit the cache for ε=0. This is the *same* workload-aware ledger as the single-table path, with only Δf changed. The unsafe envelope is rejected *before* any budget is spent: OUTER/CROSS joins, queries with more than one join, SUM/AVG over a join, and any join whose $d_{\max}$ is not a declared public schema constant (whose global sensitivity would otherwise be unbounded). An end-to-end check confirms the live `orders`⋈`lineitem` COUNT runs with Δf=7, returns $\sim 4\times$ the single-table count, caches repeats for free, and refuses every unsafe form; the behavior is covered by the unit-test suite of 233 tests (no regressions), including the budget-aware re-release ceiling of Proposition 7 and the brute-force soundness of the equivalence-checked cache. The contribution is therefore more than a purely analytical wrapper: the before-execution forecast and the join transfer are both realized in a running, tested DP-SQL system.

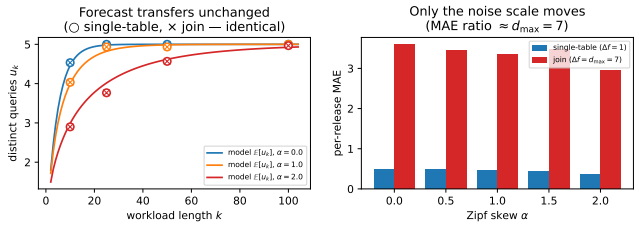

Figure 5: Twin single-table vs. join experiment (TPC-H, m=5 priority pool, $d_{\max}$=7, 600 trials). *Left:* the realized distinct count for the single-table (○) and join (×) workloads lands on the *same* analytical $\mathbb{E}[u_k]$ curve—the forecast transfers unchanged. *Right:* only the per-release noise scale moves, by Δf (1 vs $d_{\max}$=7).

Finally, because the public $d_{\max}$ clamp is deliberately conservative, we also implement an instance-optimal, data-dependent sensitivity bound computed at runtime and quantify its effect on the forecast. Following FLEX [27], the elastic sensitivity of the equi-join COUNT is $\mathrm{ES}(0) = \max(\mathrm{mf}_L, \mathrm{mf}_R)$, where mf_L and mf_R denote the maximum join-key frequencies of the left and right relations, computed from the data at *runtime*; its smooth version yields an (ε, δ) noise scale. We verify by brute force over 50k constructed join instances that $\mathrm{ES}(0)$ upper-bounds the true local sensitivity, so the mechanism is sound. For forecasting, the key observation is that the occupancy and spend are sensitivity-agnostic (Proposition 11): the forecast is *identical* whether the noise uses the public $d_{\max}$ or the runtime elastic bound, since only the per-release noise scale moves, not u_k. Consequently, the before-execution budget forecast stays exact even when the per-release noise is determined from the data at runtime. We confirm this directly: across a twin sweep the forecast quantities ($\mathbb{E}[u_k]$, $S(k)$, and the spend) are identical in every cell under the public $d_{\max}$ and the runtime elastic calibration, while only the per-release noise scale moves. The utility effect is local: on TPC-H `orders⋈lineitem` the elastic bound coincides with $d_{\max}$=7 (the realized max-frequency already equals the public cap, so there is nothing to gain), whereas on a join whose realized degree sits well below its public bound, such as `part⋈lineitem` (frequency 57 against a clamp above 114), it cuts the noise. Wiring residual sensitivity [25] or R2T [26] for multi-way and self-joins is the natural next mechanism; the forecast layer needs no change for any of them.

6 Limitations and Open Challenges

Rather than listing extensions, we pair each remaining limitation with the research problem it raises and a plausible path forward.

From repetition structure to linear structure. The deepest limitation is the unit of accounting itself. Throughout this paper a query is either a repeat of an earlier one (free), a provably equivalent rewriting of one (free, Section 3.2), or entirely new (charged in full). Real analytical workloads, however, *overlap* without being equivalent: range predicates nest, rollups aggregate the same cells at coarser granularity, and sliding windows shift by one bucket. Our forecast cannot price this overlap—two overlapping range counts form two equivalence classes and are charged twice, although much of the second answer is already determined by the first. This raises the central open problem: extending before-execution budget forecasting from *repetition* structure to *linear* structure. The natural formalization treats each aggregate as a linear query over a binned data vector x, so a workload of k queries is a matrix W_k, and the budget-relevant quantity is no longer the distinct count u_k but the dimension of the span, $\mathrm{rank}(W_k)$. A new query inside the span of the already-answered queries can be reconstructed as a linear combination of their noisy answers—by post-processing, at zero marginal privacy cost, though the reconstruction error grows with the combination coefficients, so "free" holds for the budget, not automatically for accuracy. A query outside the span decomposes as $q = q_{\parallel} + q_{\perp}$; only the orthogonal component $q_{\perp}$ carries new information and needs fresh noise calibrated to its own sensitivity—a sensitivity that projection does not automatically shrink, since the largest coefficient of $q_{\perp}$ can exceed that of q, so the decomposition saves budget on the in-span part rather than guaranteeing a cheaper fresh release. Batch versions of this idea are the matrix mechanism [10] and HDMM [32], which answer a *known* workload through an optimized strategy matrix, and Turbo [13] realizes the run-time form of the reuse: its cache, built on private multiplicative weights, answers new linear queries from the accumulated history at a fraction of the privacy cost—which is exactly the reactive counterpart of the forecast we seek. The open problem our work poses is the *before-execution* analogue of both: predicting $\mathbb{E}[\mathrm{rank}(W_k)]$, the expected dimension of the span of a random workload, the way Proposition 1 predicts $\mathbb{E}[u_k]$. Our occupancy results are exactly the special case of non-overlapping queries, where $\mathrm{rank}(W_k) = u_k$ (distinct queries with pairwise disjoint, nonzero supports are linearly independent, and a repeat changes neither side). A promising path combines the two regimes: forecast the span of the predicted workload offline and spend a budget tranche on an optimal basis for it (as the learning-augmented LAPRAS does for a predicted query set [29]), then charge only out-of-span components at run time, tracked by an online orthogonal decomposition. Practically, this requires the data domain to be binned into a compact histogram (or a low-rank or wavelet approximation) so that span tracking remains tractable.

Remaining limitations. Controlled baselines. Our comparison against reactive DP caches cites their published figures rather than a same-hardware run, because the systems are largely unavailable or unmaintained: the implementation of the academic PrivateSQL prototype was never publicly released and Chorus is no longer maintained, while the CacheDP research artifact is public but requires porting our SQL workloads to its interface. This leaves open the problem of a controlled head-to-head; the practical path is Turbo [13] or the CacheDP artifact [14]—porting our workloads to their linear-query interfaces would yield the missing benchmark, at the cost of translating SQL aggregates into their query representations.

Public key domains. GROUP BY releases assume a public, fully enumerated key domain; on sensitive or unbounded domains, releasing only the keys present leaks membership through singleton groups. The open problem is private key discovery. The standard path is a stability-based threshold, as deployed in Google's DP SQL engine [36]: noise each group count, release only groups whose

noisy count clears a threshold τ, and account the release under (ε, δ)-DP, where δ absorbs the probability that a rare key surfaces. This costs a δ and suppresses genuinely small groups, but it removes the enumeration requirement; the forecast itself is unaffected, since suppression changes what is released, not how many distinct queries are charged.

SQL coverage. The parser deliberately rejects subqueries, HAVING, DISTINCT aggregates, and window functions to keep every admitted construct's sensitivity provable; production engines such as Google's DP SQL [36] cover a broader dialect. Widening coverage without breaking the sensitivity envelope is a per-construct effort (each new construct needs its own contribution bound); the forecast layer is indifferent to the outcome, since it prices query identities, not query features.

Instance-optimal joins. The live join path uses the public $d_{\max}$ clamp, with a runtime elastic bound implemented and verified sound (Section 5); the instance-optimal mechanisms, residual sensitivity [25] and R2T [26], are discussed but not integrated into the core evaluation. Wiring one of them behind the same cache, switching the ledger to (ε, δ), and measuring the utility gain on multi-way and self-joins is the concrete next mechanism; the forecast needs no change for any of them.

Temporal accuracy. The budget ceiling is proven (Propositions 6 and 7) and the optimal refresh rate $\tau^\star(\lambda)$ follows from the boundary argument after Proposition 7, but the *magnitude* of the temporal spend is bounded only under the Poisson update model. The open problem is an accuracy theory under general renewal arrivals; a renewal-reward analysis of the re-noising process is the natural tool. Related refinements—re-noising stale entries on budget surplus, and a SUM+COUNT decomposition of AVG—would tighten utility within the same framework, and replacing basic composition with Rényi DP [16] or zCDP [17] composes with the caching, since tighter accounting lowers the per-release rate while caching lowers the number of paid releases.

Workload knowledge and scheduling. The forecast reads a declared or warmup-estimated $\{p_i\}$; the drift-aware learner and the latent-state occupancy (Proposition 10) track change, but a zero-input workload-discovery layer remains open. The deployment path runs through budget *schedulers*: in multi-tenant settings the budget is a scheduled resource (DPF [33], Cohere [34], DPack [35]; see [37] for the consolidated privacy-as-a-resource view), and these systems take per-task demands as *inputs*. Feeding them our per-workload forecasts as demand estimates would turn the planner into an admission controller, which is precisely the "forecast-driven scheduling" a production deployment needs.

External validity. Redbench is trace-derived synthesis; the arrival-process side of the model is closed (Propositions 8, 9, 10), so what remains is a second, independent production trace and an end-to-end run of raw production SQL through a DP engine.

7 Discussion and Conclusion

The experimental results support four findings. (1) Budget consumption is a function of $\mathbb{E}[u_k]$, not of the cache implementation: $\mathbb{E}[\varepsilon_{\text{wa}}] = \varepsilon_q \mathbb{E}[u_k]$ holds within $< 3\%$ in 21/24 cells, exactly on the mixed workload, and is invariant to scale and ε_q—so a deployment can estimate its privacy cost *before* running a query. (2) The folklore $1 - 1/k$ rule is recovered as the high-skew corner of a single model. (3) The per-query calibration is clean: the single-query membership-inference attack stays at or below the $e^\epsilon/(1+e^\epsilon)$ accuracy bound (within 1% for $\varepsilon \le 1$) and reconstruction tracks $1.5/\varepsilon$, while the workload-level shadow MIA was inconclusive at our sample size, a negative experimental result rather than a leakage claim. (4) The model also predicts where it fails. On *unique* workloads (and uniform over a large or growing support) savings vanish and reconstruction succeeds, exactly as predicted; on a small finite pool, a uniform workload still saturates at $m\varepsilon_q$ and continues to yield savings.

These findings have a direct practical implication. Because $\mathbb{E}[u_k]$ is computable before any query runs and is invariant to data scale and ε_q, the custodian of the dashboard scenario can size ε_q *a priori*: the safe choice $\varepsilon_q = B/m$ provably finishes the month in the static regime (since $u_k \le m$; under temporal re-noising this becomes a per-window guarantee), and the predictive allocator pushes accuracy further when the workload is skewed. The model also tells the custodian when caching cannot help—a drill-down workload spends like naive composition—so the privacy/utility budget can be planned, not discovered after exhaustion. At a *fixed* ε_q, budgeting and attack-resistance improve together in the high-skew regime. This is not automatic: if the allocator exploits skew to *raise* ε_q (e.g. B/u_k with small u_k), per-release leakage rises, so the planner must weigh the two.

Threats to validity. We discuss construct, internal, and external validity in turn, together with the role of the composition theorem. *Construct*: the headline "$\mathbb{E}[u_k]$ within $< 3\%$" is a Monte-Carlo *self-consistency* check. The empirical u_k is sampled from the very Zipf distribution the closed form integrates, so the mean converges to $\mathbb{E}[u_k]$ *by construction*; it validates the estimator's arithmetic, not the claim that real workloads are i.i.d. Zipf. We therefore stress the *deployable* forecaster out-of-distribution (Section 4.2, finding 5): estimating the workload from a 50% prefix and predicting the full u_k on five families it is never told—two Zipf, plus uniform, lognormal, and a heavy Pareto tail—the smoothed Good–Toulmin estimator stays within $\approx 2\%$ (u_k/k) across all of them, which rules out the Zipf self-consistency setup as the source of the accuracy. *Internal*: the forecast assumes a non-adaptive analyst whose choices are independent of the released noise, and we measure this rather than only assume it. When we drive a genuinely noise-adaptive analyst, one that reads each released noisy count and picks its next query from it, through the *live* middleware, and additionally stress correlated arrivals from a sticky/anti-sticky Markov process, the realized distinct count stays $u_k \le m$ in *every* policy and trial, so the ε_q=B/m allocator spends $\le B$ and never rejects, even against an adversary that couples its queries to the DP noise. The forecast itself is exact for the non-adaptive baseline (u_k=19.9 vs. $\mathbb{E}[u_k]$=19.6), conservative under positive correlation (it over-predicts u_k by up to 3.8×, the safe direction), and optimistic only under deliberately spreading policies. In no case is the privacy bound broken; only the average-case planning estimate is affected. We also test the adversarial "hunting for a specific result" case: a worst-case target-seeking analyst that steers each query by the largest released noisy count (verified to be noise-coupled, in that holding the query order fixed and varying only the Laplace draw changes the realized u_k)

makes the forecast optimistic by +12.5% (realized u_k=22.0±0.23 vs. $\mathbb{E}[u_k]$=19.6, 40 seeded trials, 95% confidence intervals), yet $u_k \le m$ and spend = $\varepsilon_q u_k \le B$ in *every* trial. Adaptivity again degrades only the planning estimate, never the budget cap. Privacy never depended on non-adaptivity, since composition is adaptive-safe and spend is hard-capped at B, and the worst-case $u_k \le m$ backstop holds under real noise-coupled adaptivity. *External*: the headline grid is synthetic, but we validate on Redbench (Section 4.2), 30 production-*derived* traces spanning 0–98% repetition with fitted Zipf $\alpha \in [0, 3]$ (mean 0.84), which raises external validity substantially. A gap remains: Redbench is trace-*derived* synthesis (real repetition structure, benchmark queries rather than raw production SQL), and the i.i.d. closed form is mildly over-optimistic about savings at the low-repetition end. A final concern is the choice of composition accounting. The forecast is *composition-agnostic*: it predicts the paid-release count u_k, the lever every composition theorem prices, so the savings equal $S(k)$ *exactly* under linear accounting (pure ε-DP and zCDP-ρ) and stay 47–78% under sub-linear advanced (ε, δ) composition. Tighter accounting changes the per-release rate, not the distinct-release count the forecast targets, so the savings are not an artifact of basic composition. The scope of the core results is single-table aggregates; the multi-table join extension is analyzed *and now executed live in the middleware* (Section 5), and instance-optimal join sensitivity (Rényi/zCDP-tight, residual-sensitivity/R2T) remains future work.

In conclusion, a closed-form model parameterized by $(\{p_i\}, k)$ predicts realized DP-budget consumption in repeated aggregate SQL workloads, recovers the folklore $1 - 1/k$ rule as a corner, and is validated to $< 3\%$ in 21/24 configurations across 4,155 trials, while flagging the regimes in which it predicts no benefit; on 30 workloads derived from real Redshift traces, forecasting from only the first half of each workload cuts the budget-prediction error by 45%. The forecast is not a bolt-on planning tool but the accounting core of the middleware, sizing its allocator, gating its re-releases, and pricing its equivalence-aware cache; Section 6 pairs each remaining limitation with the open problem it raises, chief among them extending the forecast from repetition structure to the linear span of overlapping queries. For reproducibility, the middleware, an interactive bilingual (EN/TR) web demo that animates the live pipeline step by step, and all experiment generators are public[1] with fixed integer seeds, so every table is reproduced by re-running its generator (a canonical subset of result CSVs is committed; the rest regenerate deterministically). Dependencies are listed in a `pyproject.toml` (minimum versions only; we do not ship an exact lockfile), so the model-validation and allocation experiments are deterministic within a fixed environment but not guaranteed bit-identical across library upgrades; the large six-sweep campaign is in any case only *distributionally* reproducible.

[1] https://github.com/canoztas/cmp653-project

Appendices

These appendices collect material that supports the main text but lies outside its core argument: the predictive budget allocator and its comparison with learned allocators (Appendix A), a-priori feasibility planning (Appendix B), a blind spot of assignment-free reward proxies (Appendix C), and a negative result on semantic caching (Appendix D). The complete implementation, an interactive web demo, and every experiment generator are public at https://github.com/canoztas/cmp653-project; the datasets are the standard public TPC-H, UCI Adult, and Redbench benchmarks.

A Predictive Budget Allocator

This appendix expands the allocation results summarized in the main text. The model is not only descriptive but also *prescriptive*: instead of fixing ε_q offline, an allocator can estimate $\hat{U} = \mathbb{E}[u_k]$ online from the empirical (Laplace-smoothed) template distribution after a short warmup and set $\varepsilon_q = B/\max(\hat{U}, 1)$, capped to the remaining budget so total spend never exceeds B. This targets $\varepsilon_q^* = B/u_k$, the offline optimum *under a uniform per-distinct-release objective* (Proposition 2; under a frequency- or importance-weighted error the optimum tilts ε toward the frequent/important templates, Appendix C). On Zipf workloads (k=100, B=10, 30 trials) the predictive allocator lowers per-query MAE (over answered queries) by up to ~17% at low skew, with point estimates of 16.8% at α=0 falling to 5.4% at α=2, and up to 32% in the tightest-budget regime B=1. The gain is only *borderline*, however: the paired t-test gives $p \approx 0.05$ at α=0, not crossing the 0.05 threshold, and $p > 0.3$ for $\alpha \geq 1$. We therefore interpret the improvement as directional rather than as a uniform advantage. The regime-independent result is instead the *safe* allocator ε_q=B/m. Because $u_k \leq m$ in the static regime, it never rejects and needs no warmup, and on the benchmark it answers all queries at fresh-release MAE 2.0 versus an ε-greedy bandit's 3.6 at the same 100% rate (Table 8). This 1.8× gap is the cost of the bandit's exploration, since B/m *is* the bandit's best-arm operating point (a relationship that holds when $m \leq k/4$).

Table 8: Online allocation policies (Zipf, m=20, k=100, B=10, 60 trials). Fresh MAE (over cache misses) is reported jointly with the answered fraction.

Policy	Fresh MAE	Answered	ε used
Naive (ε_q=B/k)	10.0	100/100	1.6
ε-greedy bandit	3.6	100/100	6.1
Safe closed form (B/m)	**2.0**	**100/100**	7.9
Oracle (B/u_k, needs forecast)	1.6	100/100	10.0

The allocator's quality hinges on predicting u_k from a warmup prefix. The default plug-in occupancy estimate uses only observed templates and so under-predicts. Horizon-aware extrapolators perform better in the realistic regime in which the warmup covers at least a third of the workload ($t = (k-n)/n \leq 2$): the Good–Toulmin (GT) family roughly halves the error, and the smoothed variant [20] controls its divergence beyond the $t \leq 1$ validity limit (Table 9). Chao1 [21] estimates total richness rather than the horizon-k count, and so over-predicts. Better u_k accuracy therefore buys a coverage/noise trade-off rather than an unconditional improvement: smoothed GT raises the answered fraction from 81% to 92% at the cost of higher per-release noise.

Table 9: u_k-prediction MAE (%) by estimator and extrapolation factor t (Zipf, m=50, k=100). Lower is better.

estimator	t=0.5	t=1	t=2	t=4
Plug-in occupancy	18	30	43	58
Good–Toulmin [19]	**8**	**17**	193	204
Smoothed GT [20]	**8**	**17**	**27**	151
Chao1 [21]	55	48	47	47

B A-Priori Feasibility Planning

Whereas Appendix A adapts ε_q online, the forecast also supports *capacity planning*: from a declared template distribution $\{p_i\}$ and horizon k, the planner can size ε_q and emit a feasibility verdict *before any query runs and before any budget is spent*. Under the safe sizing $\varepsilon_q = B/m$ (static-regime completion guaranteed since $u_k \leq m$), the per-release accuracy is governed by $\mathbb{P}[\,|\eta| \leq \alpha\,] = 1 - e^{-\alpha\varepsilon_q/\Delta f}$, so the planner emits a *per-release* (α, β) verdict, namely "does *each* release *individually* meet |noisy − true| $\leq \alpha$ with probability $\geq 1-\beta$?" (the standard per-query (α, β)-accuracy notion), as PASS iff $1 - e^{-\alpha\varepsilon_q/\Delta f} \geq 1 - \beta$. This is *not* the much stronger joint event that *all* releases succeed at once, which would require the per-release success probability p to satisfy $p^{u_k} \geq 1 - \beta$ (e.g. $\alpha \approx 9$ here), since under independence the all-succeed probability is the *product* of the per-release success probabilities rather than a union bound. The experiment measures exactly this per-release reading, namely the *fraction* of releases meeting the accuracy service-level agreement (SLA) rather than the all-succeed event; on a Zipf workload (m=20, k=100, B=10, β=0.2, 200 trials) the t=0 verdict matches that realized fraction (FAIL at α=2, PASS at $\alpha \geq 4$), at 100% completion. To our knowledge, no deployed system emits *this* verdict at t=0: reactive remaining-budget estimators are undefined at zero history, and accuracy-aware explorers such as APEx [8], which *do* take a per-query accuracy target and return the minimum ε that meets it, decide one query at a time, reactively, with no a-priori workload-*completion* verdict. The novel element is precisely this narrow combination: a workload-completion feasibility verdict computed at t=0 from the public support, before any budget is spent. The limits of the approach are equally explicit. Under an *under-declared* support a reactive estimator self-corrects and eventually answers more queries than our planner; under an *unbounded* horizon a position-decaying ε schedule outlasts it; and with no repetition the plan collapses to naive composition. All three failure regimes are reported in the experiment alongside the positive result.

C A Blind Spot of Assignment-Free Reward Proxies

A natural alternative to the closed-form allocator of Appendix A is to learn the allocation from reward feedback. In DP-SQL, however, the per-query error is exactly what privacy hides, so a reward-driven allocator cannot observe its true objective. Consider the weakest common surrogate: an *assignment-free* reward that sees

only the sorted granted-ε multiset and the rejection count, blind to which template each ε backs. On a weighted dashboard (one KPI at weight 10, three tiles at weight 1), two allocations with an *identical* such proxy differ by 2.71× in realized weighted error on the live middleware (0.188 vs. 0.509, 400 trials), so a learner restricted to that proxy cannot prefer the good one. This result does not imply that allocation cannot be learned. A *contextual* bandit that sees the query identity and the public weights is not blind; it can tilt ε exactly as our closed form does ($\varepsilon_i \propto \sqrt{w_i}$). The narrow point is that the information that closes the gap is the *public weights*: a learner must be *given* the same public structure as context, so that it computes the allocation rather than discovering it from a reward, and under flat weights the gap vanishes.

D A Negative Result: Semantic Caching

The cache in the main text serves only exact, equivalence-checked repeats, which raises the question of whether structurally *similar* queries that miss it could also be matched. We tested such a semantic cache: candidate matches were scored with a Collins–Duffy tree kernel [24] and a cosine similarity over abstract-syntax-tree (AST) sentence embeddings, and a match was admitted only when both scores exceeded conservative thresholds. The semantic cache fails in both directions. It admits false positives: queries differing only in WHERE literals have near-identical ASTs but different true answers, so a reused cached value is wrong by thousands of count units. It also misses textbook equivalences such as commutative AND reordering and `age>=30` vs. `age>29`, because ordered-child kernels and text embeddings do not encode logical equivalence. Returning a cached answer for a non-equivalent query does not break privacy, since reusing a DP output is still post-processing and costs zero extra budget regardless of which query it is returned for, but it is simply wrong: the answer belongs to a different query. The failure is one of correctness, not privacy. These failures show that tree-kernel and embedding similarity is not logical equivalence: a safe semantic cache needs a symbolic equivalence *prover* rather than a similarity score. A sound first step is inexpensive and exact: each query is canonicalized to a normal form before hashing, by rewriting integer comparisons to a common operator ($\texttt{age} \geq 30 \equiv \texttt{age} > 29$), orienting the column on the left-hand side, sorting commutative AND/OR conjuncts, merging redundant single-column ranges (collapsing $\texttt{age} \geq 30 \wedge \texttt{age} \geq 40$ to $\texttt{age} \geq 40$ and every provably unsatisfiable conjunction to a single canonical FALSE), and α-renaming single-table aliases; the cache is then keyed on that normal form. Two queries then collide only when provably equivalent, so the cache admits the textbook equivalences the kernel missed with zero false positives. There is no correctness loss, and the budget saving extends to literal- and ordering-variant repeats, not only byte-identical ones. We implement this normalization layer as a sound drop-in for the cache's equivalence check and confirm it end-to-end on the live middleware: on a real Adult workload it serves `age>=30` and an AND-reordered variant free as provable equivalents of `age>29` (integer columns read from the schema, so the $>$ /$\geq$ tightening is applied only where valid), while correctly refusing the non-equivalent `age>=31`, for 3 paid and 2 free over the five-query scenario, with no wrong-answer reuse. The normalizer's soundness requirement, that only provably equivalent queries collide, is checked by brute force over 60,000+ adversarial random predicate pairs (AND/OR trees, multiple columns, disequalities), evaluated directly over the integer domain, with zero false collisions. Equivalences beyond decidable predicate normalization are decided with an optional SMT layer built on the Z3 solver: a cache hit is admitted only when the solver proves the two WHERE predicates equivalent, catching integer arithmetic (`age*2>=60` ≡ `age>=30`), disjunctions of ranges, and De Morgan rewrites that the syntactic normalizer cannot (3 free SMT hits over a five-query live scenario), with a brute-force soundness fuzz test finding zero false collisions. Columns are modeled as unbounded integers, so an equivalence is admitted only if it holds for all integers (a sound subset); only general join-level equivalences remain out of scope.

References

[1] C. Dwork, F. McSherry, K. Nissim, A. Smith. Calibrating Noise to Sensitivity in Private Data Analysis. *TCC*, 2006.
[2] C. Dwork, A. Roth. The Algorithmic Foundations of Differential Privacy. *Found. Trends TCS*, 2014.
[3] I. Dinur, K. Nissim. Revealing Information While Preserving Privacy. *PODS*, 2003.
[4] F. McSherry. Privacy Integrated Queries (PINQ). *SIGMOD*, 2009.
[5] I. Kotsogiannis et al. PrivateSQL: A Differentially Private SQL Query Engine. *PVLDB* 12(11), 2019.
[6] N. Johnson, J. P. Near, J. M. Hellerstein, D. Song. Chorus: a Programming Framework for Building Scalable Differential Privacy Mechanisms. *IEEE EuroS&P*, 2020.
[7] Y. Yu et al. DOP-SQL: Differentially Private Optimization for SQL. 2024.
[8] C. Ge, X. He, I. F. Ilyas, A. Machanavajjhala. APEx: Accuracy-Aware Differentially Private Data Exploration. *SIGMOD*, 2019.
[9] X. Zhang et al. BGTplanner: Maximizing Training Accuracy for Differentially Private Federated Recommenders via Strategic Privacy Budget Allocation. *IEEE Trans. Services Computing* 18(6), 2025, 3523–3531.
[10] C. Li, G. Miklau, M. Hay, A. McGregor, V. Rastogi. The Matrix Mechanism: Optimizing Linear Counting Queries under Differential Privacy. *VLDB Journal* 24(6), 2015.
[11] W. Dong, J. Sun, K. Yi. Better than Composition: How to Answer Multiple Relational Queries under Differential Privacy. *SIGMOD*, 2023.
[12] D. Pujol, A. Sun, B. Fain, A. Machanavajjhala. Multi-Analyst Differential Privacy for Online Query Answering. *PVLDB* 16(4), 2022.
[13] K. Kostopoulou, P. Tholoniat, A. Cidon, R. Geambasu, M. Lécuyer. Turbo: Effective Caching in Differentially-Private Databases. *SOSP*, 2023.
[14] M. Mazmudar, T. Humphries, et al. Cache Me If You Can: Accuracy-Aware Inference Engine for Differentially Private Data Exploration. *PVLDB* 16(4), 2022.
[15] M. Abadi et al. Deep Learning with Differential Privacy. *CCS*, 2016.
[16] I. Mironov. Rényi Differential Privacy. *CSF*, 2017.
[17] M. Bun, T. Steinke. Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds. *TCC*, 2016.
[18] P. Flajolet, D. Gardy, L. Thimonier. Birthday Paradox, Coupon Collectors, Caching Algorithms and Self-Organizing Search. *Discrete Applied Math.* 39(3), 1992.
[19] I. J. Good, G. H. Toulmin. The Number of New Species, and the Increase in Population Coverage, when a Sample is Increased. *Biometrika* 43, 1956.
[20] A. Orlitsky, A. T. Suresh, Y. Wu. Optimal Prediction of the Number of Unseen Species. *PNAS* 113(47), 2016.
[21] A. Chao. Nonparametric Estimation of the Number of Classes in a Population. *Scand. J. Statistics* 11(4), 1984.
[22] L. Ma et al. Query-based Workload Forecasting for Self-Driving Database Management Systems (QueryBot 5000). *SIGMOD*, 2018.
[23] S. Krid, M. Stoian, A. Kipf. Redbench: A Benchmark Reflecting Real Workloads. *aiDM @ SIGMOD*, 2025. arXiv:2506.12488. (Workloads synthesized from Amazon Redshift's Redset query traces.)
[24] M. Collins, N. Duffy. Convolution Kernels for Natural Language. *NeurIPS*, 2001.
[25] W. Dong, K. Yi. Residual Sensitivity for Differentially Private Multi-Way Joins. *SIGMOD*, 2021.
[26] W. Dong, J. Fang, K. Yi, et al. R2T: Instance-optimal Truncation for Differentially Private Query Evaluation with Foreign Keys. *SIGMOD*, 2022.
[27] N. Johnson, J. P. Near, D. Song. Towards Practical Differential Privacy for SQL Queries (FLEX). *PVLDB* 11(5):526–539, 2018.
[28] K. Nissim, S. Raskhodnikova, A. Smith. Smooth Sensitivity and Sampling in Private Data Analysis. *STOC*, 2007.
[29] P. Mundra, A. Sealfon, Z. Sun, Q. C. Liu. LAPRAS: Learning-Augmented Private Answering for Linear Query Streams. arXiv:2605.01960, 2026.

[30] S. Peng, Y. Yang, Z. Zhang, M. Winslett, Y. Yu. Query Optimization for Differentially Private Data Management Systems (Pioneer). *ICDE*, 2013.
[31] J. Acharya, G. Kamath, Z. Sun, H. Zhang. INSPECTRE: Privately Estimating the Unseen. *ICML*, 2018.
[32] R. McKenna, G. Miklau, M. Hay, A. Machanavajjhala. Optimizing Error of High-Dimensional Statistical Queries under Differential Privacy. *PVLDB* 11(10), 2018.
[33] T. Luo, M. Pan, P. Tholoniat, A. Cidon, R. Geambasu, M. Lécuyer. Privacy Budget Scheduling. *OSDI*, 2021.
[34] N. Küchler, E. Opel, H. Lycklama, A. Viand, A. Hithnawi. Cohere: Managing Differential Privacy in Large Scale Systems. *IEEE S&P*, 2024.
[35] P. Tholoniat, K. Kostopoulou, et al. DPack: Efficiency-Oriented Privacy Budget Scheduling. *EuroSys*, 2025.
[36] R. J. Wilson, C. Y. Zhang, W. Lam, D. Desfontaines, D. Simmons-Marengo, B. Gipson. Differentially Private SQL with Bounded User Contribution. *PoPETs* 2020(2).
[37] P. Tholoniat. Architecting Privacy as a Computing Resource in Data Aggregation Workloads: From Browser to Datacenter. Ph.D. dissertation, Columbia University, 2025. https://doi.org/10.7916/dg57-7n35